

# EXAMEN DE PRUEBA SQL

Soluciones completas con explicación línea a línea

Plataforma  
SQL Server 2025 · SSMS 22

Base de datos  
CierzoDB · Grupo Cierzo

Contenido  
15 preguntas resueltas · Código T-SQL · Explicación de cada línea · Justificación del enfoque · Comprobaciones esperadas

## Objetivo de esta guía

No se limita a presentar consultas válidas: explica qué hace cada línea y por qué la técnica elegida encaja con el requisito del enunciado. El foco es aprender a reconocer el patrón correcto y poder adaptarlo en el examen.

## Antes de ejecutar las soluciones

Ejecuta primero el anexo del examen original para crear y cargar CierzoDB. Después trabaja en esa base de datos y ejecuta el código de cada pregunta. Las instrucciones GO son separadores de lotes que entiende SSMS; no forman parte del lenguaje T-SQL enviado al motor.

# Índice

La práctica avanza desde consultas y subconsultas hasta diseño de vistas, tablas materializadas y recorridos recursivos.

## Consultas y subconsultas

Pregunta 1. JOIN y filtros combinados

Pregunta 2. Subconsulta escalar para valores atípicos

Pregunta 3. Subconsultas anidadas sin JOIN

## CTE y recursividad

Pregunta 4. CTE para calcular cobros parciales

Pregunta 5. CTE encadenadas por etapas

Pregunta 6. CTE recursiva para aplanar unidades

## Vistas

Pregunta 7. Vista de cuenta de resultados

Pregunta 8. Vista de presupuesto frente a gasto real

Pregunta 9. Vista de exposición y riesgo de crédito

## SELECT INTO

Pregunta 10. SELECT INTO: copia estática de facturas del T3

Pregunta 11. SELECT INTO: facturas enriquecidas con estado de cobro

## Tablas temporales

Pregunta 12. Tabla temporal para reutilizar los cobros del T3

Pregunta 13. Depuración de un lote con tablas temporales

## Decisiones de diseño

Pregunta 14. CTE recursiva para localizar subordinados del CFO

Pregunta 15. Fotografía persistente de cobros a cierre del T3

## Reglas comunes que se reutilizan

- Periodo T3: fecha  $\geq$  20260701 y fecha  $<$  20261001. Se incluye el 1 de julio y se excluye el 1 de octubre.
- Cobrado por factura: SUM(cobros.importe), agrupado por factura\_id.
- Saldo pendiente: importe de factura - importe cobrado; si no existe ningún cobro, COALESCE(cobrado, 0).
- Saldo vencido: saldo pendiente de facturas con fecha\_vencimiento anterior a 2026-09-30 (comparación estricta).
- Vista: consulta guardada que refleja los datos actuales. SELECT INTO: tabla persistente creada y cargada con una copia del resultado. #tabla: tabla temporal local a la sesión.

## Pregunta 1 · JOIN y filtros combinados

Qué pide: Encontrar facturas del T3 emitidas a clientes fuera de España por un importe de al menos 80.000 €.

### Código resuelto

```
SELECT
    f.factura_id,
    f.fecha_emision,
    c.razon_social,
    c.pais,
    u.nombre AS unidad,
    cu.nombre AS cuenta,
    f.importe
FROM dbo.facturas AS f
INNER JOIN dbo.clientes AS c ON c.cliente_id = f.cliente_id
INNER JOIN dbo.unidades AS u ON u.unidad_id = f.unidad_id
INNER JOIN dbo.cuentas AS cu ON cu.cuenta_id = f.cuenta_id
WHERE f.fecha_emision >= '20260701'
      AND f.fecha_emision < '20261001'
      AND c.pais <> N'ES'
      AND f.importe >= 80000
ORDER BY f.importe DESC;
```

### Explicación línea a línea

Los números siguen el orden de las líneas del bloque de código anterior; las líneas G0 también cuentan.

1. Inicia la proyección: define qué columnas van a aparecer en el resultado.
2. Selecciona el identificador de la factura desde el alias f.
3. Incluye la fecha de emisión, necesaria para reconocer el periodo.
4. Añade la razón social del cliente, procedente de la tabla de clientes.
5. Añade el país del cliente para poder mostrar y comprobar el filtro internacional.
6. Devuelve el nombre de la unidad organizativa con el alias pedido unidad.
7. Devuelve el nombre de la cuenta contable con el alias cuenta.
8. Devuelve el importe de la factura; es el campo usado para filtrar y ordenar.
9. La tabla principal es facturas; f es un alias corto para referenciarla.
10. Relaciona cada factura con su cliente mediante la clave cliente\_id.
11. Relaciona la factura con la unidad que la emitió.
12. Relaciona la factura con la cuenta contable correspondiente.
13. Incluye emisiones desde el 1 de julio de 2026, inicio del T3.
14. Excluye el 1 de octubre y posteriores; así queda incluido todo septiembre.
15. Elimina clientes españoles. N indica literal Unicode de SQL Server.
16. Conserva facturas de 80.000 € o más; el umbral es inclusivo.
17. Ordena desde el mayor importe al menor, tal como solicita el enunciado.

### Por qué esta solución

La factura contiene las claves pero no los nombres legibles del cliente, la unidad y la cuenta. Por eso se necesitan tres JOIN. El intervalo semiabierto (inicio incluido y siguiente día excluido) evita errores habituales de límites de fecha y se reutiliza bien si la columna pasara a ser datetime.

### Comprobación esperada

La salida del enunciado contiene 5 facturas, ordenadas por importe descendente; el mayor importe es 128.000 €.

## Pregunta 2 · Subconsulta escalar para valores atípicos

Qué pide: Listar las facturas cuyo importe es estrictamente superior al doble del promedio de todas las facturas.

### Código resuelto

```
SELECT
    f.factura_id,
    c.razon_social,
    f.fecha_emision,
    f.importe
FROM dbo.facturas AS f
INNER JOIN dbo.clientes AS c ON c.cliente_id = f.cliente_id
WHERE f.importe > 2 * (
    SELECT AVG(f2.importe)
    FROM dbo.facturas AS f2
)
ORDER BY f.importe DESC;
```

### Explicación línea a línea

Los números siguen el orden de las líneas del bloque de código anterior; las líneas G0 también cuentan.

1. Comienza la lista de columnas que se devolverán.
2. Identificador de la factura.
3. Nombre legal del cliente asociado.
4. Fecha de emisión.
5. Importe que se compara con el promedio.
6. Usa facturas como tabla de referencia para calcular los atípicos.
7. Obtiene el nombre del cliente a partir de la clave foránea.
8. Exige un importe mayor que dos veces el valor escalar calculado dentro del paréntesis.
9. La subconsulta calcula la media aritmética de los importes de facturas.
10. Lee todas las facturas, sin filtros: el promedio es global.
11. Cierra la subconsulta; AVG devuelve un único valor, por lo que es una subconsulta escalar.
12. Muestra primero las facturas de importe más alto.

### Por qué esta solución

La petición compara cada factura con una única cifra de referencia: el promedio global. AVG produce un valor y no hace falta agrupar por cliente ni por periodo. El operador es > y no >= porque el enunciado dice “supera el doble”. El alias f2 separa la tabla usada para el promedio de la factura evaluada en la consulta exterior.

### Comprobación esperada

La salida esperada contiene 9 facturas. La más alta es la factura 42, de 211.200 €.

## Pregunta 3 · Subconsultas anidadas sin JOIN

Qué pide: Agrupar los cobros por método cuando el gestor del cliente tiene el puesto Comercial, sin escribir JOIN.

## Código resuelto

```
SELECT co.metodo,
       COUNT(*) AS cobros,
       SUM(co.importe) AS importe_cobrado
FROM dbo.cobros AS co
WHERE co.factura_id IN (
  SELECT f.factura_id
  FROM dbo.facturas AS f
  WHERE f.cliente_id IN (
    SELECT cl.cliente_id
    FROM dbo.clientes AS cl
    WHERE cl.gestor_id IN (
      SELECT e.empleado_id
      FROM dbo.empleados AS e
      WHERE e.puesto = N'Comercial'
    )
  )
)
GROUP BY co.metodo
ORDER BY importe_cobrado DESC;
```

## Explicación línea a línea

Los números siguen el orden de las líneas del bloque de código anterior; las líneas GO también cuentan.

1. Selecciona el método de cobro, que será también la clave de agrupación.
2. Cuenta las filas de cobros que cumplen los filtros para cada método.
3. Suma el importe cobrado en cada grupo.
4. La consulta exterior parte de cobros, que es donde están método e importe.
5. Filtra cobros a facturas cuyos identificadores devuelva la primera subconsulta.
6. Obtiene identificadores de facturas que pertenecen a clientes seleccionados.
7. La primera subconsulta consulta las facturas.
8. Restringe esas facturas a clientes incluidos en la siguiente subconsulta.
9. Devuelve los identificadores de los clientes que cumplen la condición del gestor.
10. La segunda subconsulta parte de clientes.
11. Comprueba si el gestor asignado aparece entre los empleados comerciales.
12. Selecciona los identificadores de empleados que cumplen el puesto.
13. La tercera subconsulta consulta la tabla de empleados.
14. Define la condición del puesto; el literal N admite Unicode.
15. Cierra la subconsulta de empleados.
16. Cierra la subconsulta de clientes.
17. Cierra la subconsulta de facturas.
18. Produce una fila por método de cobro.
19. Ordena los métodos desde el mayor importe total al menor.

## Por qué esta solución

La relación lógica recorre cobros → facturas → clientes → empleados. Cada IN devuelve un conjunto de claves que restringe el nivel exterior. Así se cumple la prohibición expresa de JOIN, aunque en código de producción un JOIN suele ser más directo de leer. COUNT cuenta cobros, no facturas; una factura puede tener más de un cobro.

## Comprobación esperada

La salida esperada contiene 3 métodos: Transferencia (228.000 €), Domiciliación (81.000 €) y Pagaré (66.000 €).

## Pregunta 4 · CTE para calcular cobros parciales

Qué pide: Obtener las facturas que ya han recibido algún cobro, pero conservan saldo pendiente.

## Código resuelto

```
;WITH cobrado_por_factura AS (  
    SELECT c.factura_id, SUM(c.importe) AS cobrado  
    FROM dbo.cobros AS c  
    GROUP BY c.factura_id  
)  
SELECT f.factura_id, cl.razon_social, f.importe, cp.cobrado,  
       f.importe - cp.cobrado AS pendiente  
FROM dbo.facturas AS f  
INNER JOIN cobrado_por_factura AS cp ON cp.factura_id = f.factura_id  
INNER JOIN dbo.clientes AS cl ON cl.cliente_id = f.cliente_id  
WHERE cp.cobrado > 0  
      AND cp.cobrado < f.importe  
ORDER BY pendiente DESC, f.factura_id;
```

## Explicación línea a línea

Los números siguen el orden de las líneas del bloque de código anterior; las líneas G0 también cuentan.

1. Inicia una CTE con nombre; el punto y coma garantiza que la instrucción anterior quede terminada.
2. Calcula el total recibido por cada factura a partir de todos sus cobros.
3. Lee las filas individuales de cobros.
4. Agrupa por factura para obtener una sola suma por factura.
5. Finaliza la definición de la CTE.
6. Selecciona id, cliente, importe original y total cobrado.
7. Calcula el saldo pendiente restando lo cobrado al importe de factura.
8. La consulta principal parte de facturas para conservar su importe original.
9. Un INNER JOIN conserva solo facturas con al menos un cobro registrado.
10. Obtiene la razón social del cliente.
11. Asegura que se haya recibido una cantidad positiva.
12. El total cobrado debe ser inferior al importe facturado: queda deuda.
13. Prioriza los mayores saldos pendientes y desempata por identificador.

## Por qué esta solución

La CTE encapsula un cálculo reutilizable: suma de cobros por factura. El INNER JOIN excluye las facturas sin cobros, y las dos condiciones del WHERE dejan solo los cobros parciales. No se debe sumar directamente importe de factura después de unir cada cobro individual sin agregarlo primero, porque una factura con varios cobros repetiría su importe.

## Comprobación esperada

La salida esperada contiene 9 facturas con cobro parcial. La mayor deuda pendiente es 102.400 € en la factura 33.

## Pregunta 5 · CTE encadenadas por etapas

Qué pide: Calcular facturación del T3 por país y segmento, mostrando grupos con al menos 200.000 €.

## Código resuelto

```
;WITH facturas_t3 AS (  
    SELECT factura_id, cliente_id, importe  
    FROM dbo.facturas  
    WHERE fecha_emision >= '20260701'  
        AND fecha_emision < '20261001'  
)  
,  
enriquecidas AS (  
    SELECT ft.importe, cl.pais, cl.segmento  
    FROM facturas_t3 AS ft  
    INNER JOIN dbo.clientes AS cl ON cl.cliente_id = ft.cliente_id  
)  
,  
por_segmento AS (  
    SELECT pais, segmento, COUNT(*) AS facturas, SUM(importe) AS importe  
    FROM enriquecidas  
    GROUP BY pais, segmento  
)  
SELECT pais, segmento, facturas, importe  
FROM por_segmento  
WHERE importe >= 200000  
ORDER BY importe DESC;
```

## Explicación línea a línea

Los números siguen el orden de las líneas del bloque de código anterior; las líneas GO también cuentan.

1. Define la primera etapa, limitada temporalmente a las facturas del T3.
2. Conserva solo las columnas necesarias para las siguientes etapas.
3. La primera etapa usa la tabla de facturas.
4. Incluye facturas emitidas desde el inicio del T3.
5. Excluye octubre, por lo que incluye todo el T3 sin solaparlo con T4.
6. Cierra la primera CTE y separa la definición de la siguiente.
7. La segunda etapa añade los atributos del cliente a cada factura del trimestre.
8. Se queda con importe, país y segmento, necesarios para agrupar.
9. Reutiliza el resultado de la primera CTE; no vuelve a leer todo el conjunto de facturas.
10. Añade país y segmento usando la clave del cliente.
11. Cierra la segunda CTE.
12. La tercera etapa resume las facturas por país y segmento.
13. Cuenta facturas y suma su importe para cada combinación.
14. Agrega el resultado enriquecido de la segunda etapa.
15. Forma un grupo por cada combinación de país y segmento.
16. Cierra la tercera CTE.
17. Devuelve las métricas resumidas.
18. Consulta la última etapa del pipeline.
19. Conserva las combinaciones cuyo total llega a 200.000 €.
20. Ordena desde la facturación agregada más alta a la más baja.

## Por qué esta solución

Las etapas nombradas separan responsabilidades: filtrar el periodo, enriquecer con dimensión de cliente y agregar. Esto hace que el razonamiento sea fácil de seguir y que cada CTE consuma el resultado de la anterior. La condición del umbral se aplica después de la agregación, porque se refiere a SUM(importe), no a una factura individual.

## Comprobación esperada

La salida esperada tiene 3 combinaciones: ES / Gran cuenta (528.000 €), MX / Gran cuenta (428.200 €) y PT / Distribuidor (214.600 €).

## Pregunta 6 · CTE recursiva para aplanar unidades

Qué pide: Obtener todas las unidades con su profundidad y una ruta de nombres desde la raíz.

### Código resuelto

```
;WITH jerarquia AS (  
    SELECT unidad_id, nombre, tipo, 0 AS nivel,  
           CAST(nombre AS nvarchar(max)) AS ruta  
    FROM dbo.unidades  
    WHERE unidad_padre_id IS NULL  
    UNION ALL  
    SELECT u.unidad_id, u.nombre, u.tipo, j.nivel + 1,  
           CAST(j.ruta + N' > ' + u.nombre AS nvarchar(max))  
    FROM dbo.unidades AS u  
    INNER JOIN jerarquia AS j ON u.unidad_padre_id = j.unidad_id  
)  
SELECT unidad_id, nombre, tipo, nivel, ruta  
FROM jerarquia  
ORDER BY ruta  
OPTION (MAXRECURSION 0);
```

### Explicación línea a línea

Los números siguen el orden de las líneas del bloque de código anterior; las líneas GO también cuentan.

1. Inicia la CTE recursiva que almacenará tanto la raíz como sus descendientes.
2. Define la fila inicial y fija la profundidad de la raíz en 0.
3. Inicia la ruta con el nombre de la raíz; nvarchar(max) permite rutas de longitud variable.
4. La raíz se obtiene de la tabla jerárquica de unidades.
5. Identifica la raíz porque no tiene padre.
6. Combina la raíz con las filas generadas recursivamente, sin eliminar duplicados.
7. Por cada hija devuelve sus datos y aumenta un nivel respecto a su padre.
8. Construye la ruta anteponiendo la ruta del padre y el separador >.
9. En cada iteración busca posibles unidades descendientes.
10. La unión recursiva encuentra las unidades cuyo padre ya está en la CTE.
11. Cierra la definición recursiva.
12. Devuelve los cinco campos solicitados.
13. Lee el conjunto completo producido en todas las iteraciones.
14. Ordena por la ruta completa, dando un recorrido alfabético por ramas.
15. Quita el límite predeterminado de 100 niveles; requiere que la jerarquía no tenga ciclos.

### Por qué esta solución

La relación unidad\_padre\_id apunta a otra fila de la misma tabla, así que la jerarquía tiene profundidad variable. Una CTE recursiva tiene un miembro ancla (la raíz) y un miembro recursivo (las hijas). Los CAST son importantes: SQL Server necesita que la columna ruta del ancla y la recursiva tenga el mismo tipo. MAXRECURSION 0 evita el tope de 100 niveles, pero una jerarquía con ciclos debe corregirse para que la recursión no continúe indefinidamente.

### Comprobación esperada

La salida esperada contiene las 17 unidades, con Grupo Cierzo en nivel 0 y los centros de coste en nivel 3.

## Pregunta 7 · Vista de cuenta de resultados

Qué pide: Crear una vista con movimientos de ingresos por factura y gastos por apunte, asignados al trimestre correspondiente.

## Código resuelto

```
CREATE OR ALTER VIEW dbo.vw_cuenta_resultados
AS
SELECT p.ejercicio, p.trimestre, cu.tipo, cu.nombre AS cuenta,
       SUM(f.importe) AS importe
FROM dbo.facturas AS f
INNER JOIN dbo.periodos AS p
  ON f.fecha_emision BETWEEN p.fecha_inicio AND p.fecha_fin
INNER JOIN dbo.cuentas AS cu ON cu.cuenta_id = f.cuenta_id
GROUP BY p.ejercicio, p.trimestre, cu.tipo, cu.nombre
UNION ALL
SELECT p.ejercicio, p.trimestre, cu.tipo, cu.nombre AS cuenta,
       SUM(g.importe) AS importe
FROM dbo.gastos AS g
INNER JOIN dbo.periodos AS p
  ON g.fecha BETWEEN p.fecha_inicio AND p.fecha_fin
INNER JOIN dbo.cuentas AS cu ON cu.cuenta_id = g.cuenta_id
GROUP BY p.ejercicio, p.trimestre, cu.tipo, cu.nombre;
GO
SELECT trimestre, tipo, SUM(importe) AS importe
FROM dbo.vw_cuenta_resultados
GROUP BY trimestre, tipo
ORDER BY trimestre, tipo;
```

## Explicación línea a línea

Los números siguen el orden de las líneas del bloque de código anterior; las líneas GO también cuentan.

1. Crea la vista si no existe o actualiza su definición si ya existe.
2. Marca el inicio de la consulta que define la vista.
3. Define las dimensiones del resultado: año, trimestre, tipo de movimiento y cuenta.
4. Suma el importe de las facturas por grupo.
5. La rama de ingresos parte de las facturas emitidas.
6. Relaciona cada factura con un trimestre contable.
7. Asigna la factura al periodo que contiene su fecha de emisión.
8. Obtiene el tipo y nombre de la cuenta de ingreso.
9. Agrega por año, trimestre y cuenta para que cada grupo tenga una fila.
10. Apila los resultados de gastos debajo de los ingresos sin eliminar filas.
11. La rama de gastos devuelve las mismas columnas y en el mismo orden.
12. Suma los apuntes de gasto para cada combinación.
13. La segunda rama parte de los gastos contabilizados.
14. Relaciona cada gasto con el trimestre que contiene su fecha.
15. El gasto se fecha por g.fecha, no por la fecha de una factura.
16. Obtiene el tipo y nombre de la cuenta de gasto.
17. Agrupa los gastos de forma equivalente a la rama de ingresos.
18. En SSMS separa el lote: CREATE OR ALTER VIEW debe ser la primera instrucción de su lote.
19. Consulta de comprobación: resume el total publicado por la vista.
20. Lee la vista, no las tablas base.
21. Calcula una cifra total por trimestre y tipo de movimiento.
22. Ordena la comprobación por trimestre y después por tipo.

## Por qué esta solución

Una vista proporciona una capa lógica estable para BI. La consulta usa UNION ALL porque ambas ramas representan hechos diferentes (facturas y gastos) y deben conservarse. Se agrupa por cuenta para que la vista tenga una fila por ejercicio, trimestre, tipo y cuenta. No se añade ORDER BY a la definición de la vista: el orden se solicita en la consulta que la consume.

## Comprobación esperada

La comprobación debe devolver 6 filas (Gasto e Ingreso para los trimestres 1, 2 y 3). Los totales esperados son: T1 gasto 1.426.490 € / ingreso 1.483.000 €; T2 gasto 1.530.590 € / ingreso 1.608.400 €; T3 gasto 1.330.600 € / ingreso 1.306.500 €.

## Pregunta 8 · Vista de presupuesto frente a gasto real

Qué pide: Crear una vista por línea presupuestada de gasto, incluyendo líneas sin gasto real y calculando la desviación.

### Código resuelto

```
CREATE OR ALTER VIEW dbo.vw_presupuesto_gastos
AS
WITH gasto_real AS (
    SELECT p.periodo_id, g.unidad_id, g.cuenta_id,
           SUM(g.importe) AS gasto_real
    FROM dbo.gastos AS g
    INNER JOIN dbo.periodos AS p
        ON g.fecha BETWEEN p.fecha_inicio AND p.fecha_fin
    GROUP BY p.periodo_id, g.unidad_id, g.cuenta_id
)
SELECT p.ejercicio, p.trimestre, u.nombre AS unidad,
       cu.nombre AS cuenta, pr.importe AS presupuesto,
       COALESCE(gr.gasto_real, 0) AS gasto_real,
       COALESCE(gr.gasto_real, 0) - pr.importe AS desviacion
FROM dbo.presupuestos AS pr
INNER JOIN dbo.periodos AS p ON p.periodo_id = pr.periodo_id
INNER JOIN dbo.unidades AS u ON u.unidad_id = pr.unidad_id
INNER JOIN dbo.cuentas AS cu ON cu.cuenta_id = pr.cuenta_id
LEFT JOIN gasto_real AS gr
    ON gr.periodo_id = pr.periodo_id
   AND gr.unidad_id = pr.unidad_id
   AND gr.cuenta_id = pr.cuenta_id
WHERE cu.tipo = N'Gasto';
GO
SELECT unidad, cuenta, presupuesto, gasto_real, desviacion
FROM dbo.vw_presupuesto_gastos
WHERE ejercicio = 2026 AND trimestre = 3 AND desviacion > 5000
ORDER BY desviacion DESC;
```

### Explicación línea a línea

Los números siguen el orden de las líneas del bloque de código anterior; las líneas GO también cuentan.

1. Crea la vista o sustituye su definición actual, facilitando ejecutar el script de nuevo.
2. Inicia la consulta que formará el contenido de la vista.
3. Define una CTE que calcula primero el gasto contabilizado por periodo, unidad y cuenta.
4. Identifica las tres claves que tienen que coincidir con cada línea de presupuesto.
5. Suma los importes reales dentro de cada grupo.
6. Lee los movimientos de gasto.
7. Asocia cada movimiento al periodo contable.
8. Busca el trimestre que contiene la fecha del gasto.
9. Produce una suma por periodo, unidad y cuenta.
10. Cierra la CTE de importes reales.
11. Añade año, trimestre y nombre legible de la unidad.
12. Añade el nombre de la cuenta y el presupuesto aprobado.
13. Si no hay coincidencia en la CTE, presenta 0 en vez de NULL.
14. Calcula gasto real menos presupuesto: positivo significa exceso de gasto.
15. La tabla de presupuestos conduce la consulta para no perder líneas presupuestadas.
16. Recupera ejercicio y trimestre de cada línea.

17. Convierte la clave de unidad en su nombre.
18. Recupera el tipo y el nombre de la cuenta.
19. LEFT JOIN conserva la línea de presupuesto aunque no exista gasto real.
20. Exige que coincida el periodo.
21. Exige que coincida la unidad.
22. Exige que coincida la cuenta.
23. Limita la vista a las cuentas de tipo Gasto.
24. Separa la definición de la vista de la consulta de comprobación en SSMS.
25. Selecciona los campos que pide la comprobación.
26. Consulta la vista creada.
27. Limita al T3 de 2026 y a excesos superiores a 5.000 €.
28. Muestra primero los mayores excesos frente al presupuesto.

## Por qué esta solución

La tabla conductora debe ser presupuestos: el enunciado exige una fila por cada línea presupuestada de gasto, incluso sin ejecución. Por eso se agrega gasto real en una CTE y se conecta mediante LEFT JOIN. COALESCE reemplaza el NULL que aparece cuando no existe movimiento; la desviación se define expresamente como real menos presupuesto.

## Comprobación esperada

La comprobación esperada devuelve 6 líneas del T3 con desviación superior a 5.000 €. La mayor es Proyectos / Servicios profesionales, con desviación de 29.000 €.

# Pregunta 9 · Vista de exposición y riesgo de crédito

Qué pide: Crear una fila por cliente, incluso si no tiene facturas, con exposición y saldo vencido al 30/09/2026.

## Código resuelto

```
CREATE OR ALTER VIEW dbo.vw_riesgo_cliente
AS
WITH cobrado_por_factura AS (
    SELECT factura_id, SUM(importe) AS cobrado
    FROM dbo.cobros
    GROUP BY factura_id
),
saldo_por_factura AS (
    SELECT f.factura_id, f.cliente_id, f.fecha_vencimiento,
           f.importe - COALESCE(cp.cobrado, 0) AS saldo_pendiente
    FROM dbo.facturas AS f
    LEFT JOIN cobrado_por_factura AS cp ON cp.factura_id = f.factura_id
)
SELECT cl.cliente_id, cl.razon_social, cl.rating, cl.limite_credito,
       COALESCE(SUM(spf.saldo_pendiente), 0) AS exposicion,
       COALESCE(SUM(CASE
           WHEN spf.fecha_vencimiento < CONVERT(date, '20260930')
           THEN spf.saldo_pendiente ELSE 0 END), 0) AS saldo_vencido
FROM dbo.clientes AS cl
LEFT JOIN saldo_por_factura AS spf ON spf.cliente_id = cl.cliente_id
GROUP BY cl.cliente_id, cl.razon_social, cl.rating, cl.limite_credito;
GO
SELECT razon_social, rating, limite_credito, exposicion, saldo_vencido
FROM dbo.vw_riesgo_cliente
WHERE exposicion > limite_credito
ORDER BY exposicion DESC;
```

## Explicación línea a línea

Los números siguen el orden de las líneas del bloque de código anterior; las líneas GO también cuentan.

1. Crea la vista o actualiza su definición si ya existe.
2. Inicia la consulta de la vista.
3. Primera CTE: suma todos los cobros recibidos para cada factura.
4. Calcula el importe total cobrado por factura.
5. Lee los movimientos de cobro.
6. Devuelve una fila agregada por factura.
7. Cierra la primera CTE y abre paso a la segunda.
8. Segunda CTE: traduce el importe facturado y cobrado a saldo pendiente.
9. Conserva las claves de factura y cliente, y la fecha necesaria para revisar vencimientos.
10. Resta los cobros y trata una factura sin cobros como cobrado 0.
11. Parte de todas las facturas emitidas.
12. Mantiene facturas sin cobros; sin coincidencia, la CTE devuelve cobrado NULL.
13. Cierra la segunda CTE.
14. Selecciona los datos de identidad y crédito del cliente.
15. Suma saldos pendientes del cliente y devuelve 0 si no hay facturas.
16. Empieza la suma condicional que compondrá solo la parte vencida de la exposición.
17. Considera vencidas solo facturas con vencimiento estrictamente anterior al corte.
18. Suma el saldo de las facturas vencidas; el resto aporta 0.
19. Se parte de todos los clientes para que la vista tenga una fila por cliente.
20. Conserva clientes sin facturas, para los que las métricas quedarán a cero.
21. Agrupa por cada atributo de cliente que aparece sin agregación.
22. Termina el lote de creación de la vista en SSMS.
23. Prepara la consulta de comprobación con las columnas relevantes.
24. Lee los cálculos centralizados en la vista.
25. Filtra clientes cuya deuda total pendiente supera su límite de crédito.
26. Ordena por exposición de mayor a menor.

## Por qué esta solución

Hay dos agregaciones conceptualmente distintas: cobros por factura y saldos por factura. Separarlas evita duplicar importes. La vista parte de clientes y usa LEFT JOIN para no perder los que no tienen facturas. La fecha de corte se aplica solo al saldo vencido y la comparación es estricta (<), tal como define el enunciado.

## Comprobación esperada

La vista debe contener los 14 clientes. La comprobación de clientes por encima de su límite devuelve 2 filas: Construcciones Peñalara S.A. y Envases Tapatíos S.A. de C.V.

# Pregunta 10 · SELECT INTO: copia estática de facturas del T3

Qué pide: Crear una tabla permanente con las facturas emitidas entre julio y septiembre de 2026 y poder repetir el script.

## Código resuelto

```
IF OBJECT_ID(N'dbo.facturas_2026_t3', N'U') IS NOT NULL
    DROP TABLE dbo.facturas_2026_t3;
SELECT factura_id, cliente_id, fecha_emision, fecha_vencimiento, importe
INTO dbo.facturas_2026_t3
FROM dbo.facturas
WHERE fecha_emision >= '20260701'
    AND fecha_emision < '20261001';
SELECT COUNT(*) AS facturas, SUM(importe) AS importe_total,
    MIN(fecha_emision) AS primera_emision,
    MAX(fecha_emision) AS ultima_emision
FROM dbo.facturas_2026_t3;
```

## Explicación línea a línea

Los números siguen el orden de las líneas del bloque de código anterior; las líneas G0 también cuentan.

1. Comprueba si ya existe una tabla de usuario con ese nombre en el esquema dbo.
2. Elimina la versión anterior para que la siguiente creación no falle por nombre duplicado.
3. Selecciona exactamente las columnas solicitadas para la copia.
4. Crea la tabla destino y la carga con el resultado del SELECT en una única instrucción.
5. Los datos proceden de la tabla original de facturas.
6. Incluye facturas desde el 1 de julio.
7. Excluye octubre y fechas posteriores.
8. La comprobación cuenta filas y suma sus importes.
9. Calcula la primera fecha de emisión almacenada.
10. Calcula la última fecha de emisión almacenada.
11. Comprueba la copia, no la tabla de origen.

## Por qué esta solución

SELECT INTO resulta adecuado para una copia de análisis porque crea la estructura básica según las columnas seleccionadas y carga las filas al mismo tiempo. La tabla creada es permanente y no se actualiza automáticamente al cambiar la tabla de origen. El DROP condicional hace repetible el ejemplo; no afecta a dbo.facturas.

## Comprobación esperada

La comprobación esperada es: 21 facturas, 1.306.500 € en total, primera emisión 2026-07-06 y última emisión 2026-09-30.

# Pregunta 11 · SELECT INTO: facturas enriquecidas con estado de cobro

Qué pide: Generar una tabla permanente con datos del cliente, cobrado, pendiente y estado para cada factura.

## Código resuelto

```
IF OBJECT_ID(N'dbo.facturas_enriquecidas', N'U') IS NOT NULL
    DROP TABLE dbo.facturas_enriquecidas;
;WITH cobrado_por_factura AS (
    SELECT factura_id, SUM(importe) AS cobrado
    FROM dbo.cobros
    GROUP BY factura_id
)
SELECT f.factura_id, cl.razon_social, cl.pais, f.fecha_emision,
       f.importe, COALESCE(cp.cobrado, 0) AS cobrado,
       f.importe - COALESCE(cp.cobrado, 0) AS pendiente,
       CASE
           WHEN COALESCE(cp.cobrado, 0) = 0 THEN N'Sin cobro'
           WHEN COALESCE(cp.cobrado, 0) < f.importe THEN N'Cobro parcial'
           ELSE N'Cobrada'
       END AS estado_cobro
INTO dbo.facturas_enriquecidas
FROM dbo.facturas AS f
INNER JOIN dbo.clientes AS cl ON cl.cliente_id = f.cliente_id
LEFT JOIN cobrado_por_factura AS cp ON cp.factura_id = f.factura_id;
SELECT estado_cobro, COUNT(*) AS facturas, SUM(importe) AS importe,
       SUM(pendiente) AS pendiente
FROM dbo.facturas_enriquecidas
GROUP BY estado_cobro
ORDER BY facturas DESC;
```

## Explicación línea a línea

Los números siguen el orden de las líneas del bloque de código anterior; las líneas G0 también cuentan.

1. Comprueba si existe la tabla destino de una ejecución anterior.
2. La elimina si existe para poder regenerarla sin error.
3. Inicia una CTE para reducir los cobros a una suma por factura.
4. Calcula el importe total cobrado para cada factura con cobros.
5. Lee la tabla de cobros.
6. Devuelve una única fila agregada por factura.
7. Cierra la CTE que después se unirá a las facturas.
8. Selecciona identificador, cliente, país y fecha de emisión.
9. Añade importe de factura y sustituye por 0 la ausencia de cobros.
10. Calcula el saldo pendiente.
11. Inicia la expresión condicional que clasifica cada factura.
12. Si no hay cobro, asigna el estado Sin cobro.
13. Si se cobró algo pero no todo el importe, clasifica como Cobro parcial.
14. Si no se cumplen las dos condiciones anteriores, queda totalmente cobrada.
15. Cierra el CASE y nombra la columna resultante.
16. Crea y carga la tabla enriquecida con el resultado del SELECT.
17. Mantiene todas las facturas como conjunto base.
18. Obtiene los atributos del cliente de cada factura.
19. Incluye también facturas sin cobros, que recibirán 0 en cobrado.
20. Inicia la comprobación con el número de facturas y su importe por estado.
21. Suma el saldo pendiente de cada grupo.
22. Usa la tabla generada.
23. Devuelve una fila por estado de cobro.
24. Muestra primero el estado que contiene más facturas.

## Por qué esta solución

El importe cobrado se agrega antes del JOIN para que una factura con varios cobros siga produciendo una sola fila. LEFT JOIN conserva las facturas impagadas. CASE es apropiado porque los estados son mutuamente excluyentes y deben seguir un orden lógico: sin ningún cobro, con cobro parcial y, por último, totalmente cobrada.

## Comprobación esperada

La comprobación esperada devuelve 38 facturas Cobradas, 21 Sin cobro y 9 con Cobro parcial. Los importes facturados por estado son 1.784.900 €, 1.666.400 € y 946.600 €, respectivamente.

# Pregunta 12 · Tabla temporal para reutilizar los cobros del T3

Qué pide: Materializar una vez los cobros del T3 y calcular tanto el resumen por método como los tres mayores clientes.

## Código resuelto

```
IF OBJECT_ID('tempdb..#cobros_t3') IS NOT NULL
    DROP TABLE #cobros_t3;
SELECT co.cobro_id, cl.razon_social, co.fecha_cobro, co.importe, co.metodo
INTO #cobros_t3
FROM dbo.cobros AS co
INNER JOIN dbo.facturas AS f ON f.factura_id = co.factura_id
INNER JOIN dbo.clientes AS cl ON cl.cliente_id = f.cliente_id
WHERE co.fecha_cobro >= '20260701'
    AND co.fecha_cobro < '20261001';
SELECT metodo, COUNT(*) AS cobros, SUM(importe) AS importe_cobrado
FROM #cobros_t3
GROUP BY metodo
ORDER BY importe_cobrado DESC;
SELECT TOP (3) razon_social, SUM(importe) AS importe_cobrado
FROM #cobros_t3
GROUP BY razon_social
ORDER BY importe_cobrado DESC;
DROP TABLE #cobros_t3;
```

## Explicación línea a línea

Los números siguen el orden de las líneas del bloque de código anterior; las líneas G0 también cuentan.

1. Comprueba si la tabla temporal local sigue creada en la sesión.
2. La elimina para que el bloque se pueda ejecutar otra vez en la misma sesión.
3. Selecciona los campos reutilizables: cobro, cliente, fecha, importe y método.
4. Crea una tabla temporal local y la rellena en el mismo paso.
5. Parte de movimientos de cobro, ya que el periodo depende de fecha\_cobro.
6. Localiza el cliente de cada cobro pasando por la factura.
7. Añade la razón social para que las consultas posteriores no repitan el JOIN.
8. Incluye cobros desde el 1 de julio.
9. Excluye cobros desde octubre en adelante.
10. Consulta A: cuenta y suma los cobros por método.
11. Usa la tabla temporal ya preparada, sin repetir cruces con las tablas base.
12. Produce un resumen por método.
13. Ordena los métodos por importe total cobrado.
14. Consulta B: selecciona los tres clientes con mayor cobro agregado.
15. Reutiliza exactamente el mismo conjunto del T3.
16. Agrega los importes de todos los cobros por cliente.
17. Ordena antes de aplicar TOP, para obtener los tres mayores.
18. Elimina explícitamente la tabla temporal después de obtener ambos resultados.

## Por qué esta solución

Las dos métricas comparten el mismo filtro de fechas y los mismos cruces; materializar el conjunto evita repetir ese trabajo y hace que ambas consultas usen la misma base. Una tabla temporal local solo es visible dentro de la sesión y se elimina al cerrar la sesión; aquí también se elimina explícitamente al terminar.

## Comprobación esperada

Consulta A: Transferencia 483.400 €, Confirming 276.800 €, Domiciliación 85.900 € y Pagaré 64.000 €.

Consulta B: Naviera Atlántida (301.800 €), Lusitana de Energia (160.600 €) y Aceros Regiomontanos (153.400 €).

# Pregunta 13 · Depuración de un lote con tablas temporales

Qué pide: Eliminar duplicados exactos del staging, validar filas y comparar el número de registros de cada etapa sin cargar dbo.cobros.

## Código resuelto

```

IF OBJECT_ID('tempdb..#lote_unico') IS NOT NULL DROP TABLE #lote_unico;
IF OBJECT_ID('tempdb..#lote_valido') IS NOT NULL DROP TABLE #lote_valido;
SELECT DISTINCT cobro_id, factura_id, fecha_cobro, importe, metodo
INTO #lote_unico
FROM dbo.stg_cobros;
SELECT lu.cobro_id, lu.factura_id, lu.fecha_cobro, lu.importe, lu.metodo
INTO #lote_valido
FROM #lote_unico AS lu
WHERE lu.importe > 0
    AND EXISTS (
        SELECT 1
        FROM dbo.facturas AS f
        WHERE f.factura_id = lu.factura_id
    )
    AND NOT EXISTS (
        SELECT 1
        FROM dbo.cobros AS co
        WHERE co.cobro_id = lu.cobro_id
    );
SELECT etapa, filas
FROM (
    SELECT N'stg_cobros' AS etapa, COUNT(*) AS filas
    FROM dbo.stg_cobros
    UNION ALL
    SELECT N'#lote_unico', COUNT(*) FROM #lote_unico
    UNION ALL
    SELECT N'#lote_valido', COUNT(*) FROM #lote_valido
) AS conteos
ORDER BY filas DESC;
SELECT cobro_id, factura_id, fecha_cobro, importe, metodo
FROM #lote_valido
ORDER BY cobro_id;
DROP TABLE #lote_valido;
DROP TABLE #lote_unico;

```

## Explicación línea a línea

Los números siguen el orden de las líneas del bloque de código anterior; las líneas G0 también cuentan.

1. Elimina una versión anterior de la primera tabla temporal, si existe.
2. Elimina una versión anterior de la tabla con filas validadas.
3. Elimina duplicados exactos considerando todas las columnas del lote.
4. Crea y carga la primera tabla temporal con las filas únicas.
5. El origen es staging: no se modifica la tabla de cobros productiva.
6. Selecciona las columnas del lote único que se conservarán si pasan validación.
7. Crea la tabla temporal del segundo paso con solo registros válidos.
8. Valida sobre el resultado sin duplicados, no sobre el staging original.
9. Rechaza importes nulos o no positivos mediante la condición estricta  $> 0$ .
10. Exige que exista la factura referenciada por el cobro.
11. La subconsulta solo necesita comprobar existencia; no necesita devolver columnas.
12. Busca la factura en el modelo de datos.
13. La existencia se verifica por el identificador de factura.
14. Cierra la validación de existencia de factura.
15. Exige que el identificador de cobro todavía no esté cargado.
16. Comprueba existencia de una fila que ya tenga ese identificador.
17. Consulta la tabla productiva de cobros únicamente para validar.
18. Si ya existe el ID, NOT EXISTS es falso y la fila se descarta.
19. Cierra la condición y termina la creación de #lote\_valido.
20. Consulta A devuelve una etiqueta de etapa y su número de filas.
21. Empieza una tabla derivada que reunirá los tres recuentos.
22. Cuenta las filas originales de staging.
23. Lee el lote tal como llegó del banco.

24. Combina el primer recuento con el siguiente sin eliminar etapas repetidas.
25. Cuenta las filas tras quitar duplicados exactos.
26. Añade el último recuento.
27. Cuenta las filas que superaron las tres validaciones.
28. Da un alias obligatorio a la tabla derivada.
29. Ordena las etapas desde el mayor recuento al menor.
30. Consulta B muestra el contenido validado.
31. Lee solamente las filas aceptadas para una futura carga.
32. Ordena las filas por identificador de cobro.
33. Limpia la segunda tabla temporal al terminar.
34. Limpia la primera tabla temporal al terminar.

## Por qué esta solución

La depuración se separa en dos fases para poder auditar cuántas filas se eliminan por cada paso. DISTINCT quita copias idénticas; EXISTS confirma que la factura existe, y NOT EXISTS evita reutilizar un cobro ya registrado. No hay INSERT ni UPDATE sobre dbo.cobros: la solución solo genera tablas temporales y consultas de diagnóstico.

## Comprobación esperada

La salida esperada compara 10 filas en stg\_cobros, 9 en #lote\_unico y 5 en #lote\_valido. Las filas válidas son los cobros 49, 50, 51, 54 y 55.

# Pregunta 14 · CTE recursiva para localizar subordinados del CFO

Qué pide: Devolver subordinados directos e indirectos de Álvaro Montes, numerando el nivel jerárquico y sin incluir al CFO.

## Código resuelto

```
;WITH subordinados AS (
    SELECT e.empleado_id, e.nombre, e.puesto, 1 AS nivel
    FROM dbo.empleados AS e
    INNER JOIN dbo.empleados AS jefe ON jefe.empleado_id = e.jefe_id
    WHERE jefe.puesto = N'CFO'
    UNION ALL
    SELECT e.empleado_id, e.nombre, e.puesto, s.nivel + 1
    FROM dbo.empleados AS e
    INNER JOIN subordinados AS s ON e.jefe_id = s.empleado_id
)
SELECT nombre, puesto, nivel
FROM subordinados
ORDER BY nivel, nombre
OPTION (MAXRECURSION 0);
```

## Explicación línea a línea

Los números siguen el orden de las líneas del bloque de código anterior; las líneas GO también cuentan.

1. Inicia una CTE recursiva para explorar empleados que reportan, en cualquier profundidad, a la cadena del CFO.
2. El ancla selecciona los subordinados directos y los marca como nivel 1.
3. La fila candidata es el empleado subordinado.
4. Relaciona cada empleado con su jefe mediante la autorrelación de la tabla.
5. Encuentra empleados cuyo jefe ocupa el puesto CFO, sin incluir al propio CFO.
6. Combina los subordinados directos con los encontrados en las iteraciones recursivas.
7. Cada nueva generación recibe el nivel del jefe más uno.
8. Busca empleados que puedan depender de alguien ya seleccionado.
9. La condición recursiva añade empleados cuyo jefe está en el conjunto actual.
10. Cierra la definición de la CTE recursiva.

11. Devuelve solo las tres columnas solicitadas.
12. Consulta todos los empleados descubiertos por la CTE.
13. Ordena primero por profundidad y luego alfabéticamente dentro de cada nivel.
14. Permite recorrer más de 100 niveles; presupone que no hay ciclos en jefe\_id.

## Por qué esta solución

La CTE recursiva es la elección correcta porque la profundidad de la jerarquía puede cambiar con el tiempo. Una cadena de subconsultas de profundidad fija exigiría modificar el código cada vez que se añadiese otro nivel de mando. El ancla devuelve informes directos; el miembro recursivo busca, sucesivamente, los informes de cada empleado encontrado.

## Comprobación esperada

La salida esperada contiene 5 empleados: Nerea Azcona y Patricia Soler en nivel 1; Clara Mendiola, Hugo Barrios e Iván Calvo en nivel 2.

# Pregunta 15 · Fotografía persistente de cobros a cierre del T3

Qué pide: Guardar el total cobrado por cliente hasta el 30/09/2026 en una tabla permanente, estable ante cobros futuros y creada/cargada con SELECT INTO.

## Código resuelto

```
IF OBJECT_ID(N'dbo.cierre_cobros_2026_t3', N'U') IS NOT NULL
    DROP TABLE dbo.cierre_cobros_2026_t3;
SELECT cl.cliente_id, cl.razon_social, SUM(co.importe) AS importe_cobrado,
       CONVERT(date, '20260930') AS fecha_foto
INTO dbo.cierre_cobros_2026_t3
FROM dbo.cobros AS co
INNER JOIN dbo.facturas AS f ON f.factura_id = co.factura_id
INNER JOIN dbo.clientes AS cl ON cl.cliente_id = f.cliente_id
WHERE co.fecha_cobro <= CONVERT(date, '20260930')
GROUP BY cl.cliente_id, cl.razon_social;
SELECT cliente_id, razon_social, importe_cobrado, fecha_foto
FROM dbo.cierre_cobros_2026_t3
ORDER BY importe_cobrado DESC;
```

## Explicación línea a línea

Los números siguen el orden de las líneas del bloque de código anterior; las líneas G0 también cuentan.

1. Comprueba si ya existe la tabla de fotografía de cierre.
2. Elimina la versión anterior para que el script pueda repetirse desde cero.
3. Selecciona la identidad del cliente y suma todos sus cobros incluidos en la foto.
4. Añade una fecha constante de fotografía, con tipo date y valor 30/09/2026.
5. Crea y rellena la tabla permanente en una sola instrucción SELECT INTO.
6. Parte de los cobros realmente recibidos.
7. Relaciona cada cobro con su factura para saber a qué cliente corresponde.
8. Obtiene cliente\_id y razón social.
9. Solo incluye cobros recibidos hasta la fecha de cierre; no incorpora cobros de octubre.
10. Agrega una única fila por cliente con su importe acumulado hasta el corte.
11. Prepara la comprobación con las columnas de la fotografía.
12. Lee la tabla persistente que ya contiene los valores congelados.
13. Presenta primero el cliente con mayor importe cobrado.

## Por qué esta solución

La elección correcta es una tabla permanente materializada con SELECT INTO. Una vista volvería a calcular importes contra dbo.cobros y cambiaría cuando se cargaran movimientos nuevos. Una tabla temporal desaparecería al cerrar la sesión y no sería común a los analistas. SELECT INTO crea la tabla física y la llena en la misma instrucción; después, los cobros nuevos no alteran automáticamente sus datos.

## Comprobación esperada

La comprobación esperada muestra 12 clientes con cobros, de mayor a menor importe: Naviera Atlántida (535.800 €), Aceros Regiomontanos (354.000 €), Lusitana de Energia (321.600 €), entre otros. Todas las filas llevan fecha\_foto = 2026-09-30.

---

## Repaso final antes del examen

**Fechas:** Usa límites de fecha explícitos. Para el T3,  $\geq 20260701$  y  $< 20261001$  evita incluir el 1 de octubre.

**Agregaciones:** Si una factura tiene varios cobros, suma primero por factura antes de unir el resultado con facturas.

**Filas sin coincidencia:** Elige LEFT JOIN si el enunciado pide conservar todas las entidades base; usa COALESCE para convertir NULL en 0 cuando proceda.

**Filtros de agregación:** Si la condición depende de SUM o COUNT, normalmente debe aplicarse después del GROUP BY, sobre el resultado agregado.

**CTE recursiva:** Define el ancla y la regla que genera el siguiente nivel. Verifica que las columnas del ancla y del miembro recursivo tengan tipos compatibles.

**Vistas:** No guardan una fotografía de los datos y no garantizan orden de salida. Ordena en la consulta que las utiliza.

**SELECT INTO:** Crea una tabla permanente y carga el resultado. Para poder relanzar el script, elimina primero la tabla previa si existe.

**Tablas temporales:** Una tabla local #tabla vive en la sesión actual; es útil para reutilizar un conjunto intermedio y luego se puede borrar.

**Validación de lotes:** DISTINCT elimina duplicados exactos; EXISTS valida referencias y NOT EXISTS comprueba que la clave aún no esté cargada.

**Elección de técnica:** Para profundidad jerárquica variable, usa CTE recursiva; para una foto persistente, usa una tabla materializada y no una vista ni una tabla temporal.

## Nota de alcance

Las soluciones se basan en el modelo, las reglas de negocio y las salidas esperadas que aparecen en el examen de Grupo Cierzo. No incluyen el anexo completo de creación de la base de datos: ese script debe ejecutarse desde el documento original antes de probar estas respuestas.